

polars & tidy data

Lecture 09

Dr. Colin Rundel



polars

polars (2020) is a newer data frame library written in Rust on top of Apache Arrow's memory layout, with bindings for Python, R, and JavaScript. Rather than extend NumPy conventions it was designed from scratch around a few ideas,

- No index - rows are only ever positions
- A strict schema - columns have explicit types and incompatible values are rejected by default; expressions can promote types or replace columns
- Expressions - computations on columns are objects, built up and then evaluated in a context
- Lazy evaluation - queries are planned and optimized before running, on all available cores

```
1 import polars as pl
2 pl.__version__
```

py

'1.44.2'

Series

A polars Series is a name, a dtype, and values - nothing else. With no index there is nothing to align on, so arithmetic is positional and lengths must match (or be length one).

```
1 pl.Series("x", [4, 2, 1, 3])
```

py

shape: (4,)

x
4
2
1
3

```
1 a = pl.Series([1, 2, 3])
2 b = pl.Series([10, 20, 30])
3 a + b
```

py

shape: (3,)

11
22
33

DataFrame

Print method shows the shape and column dtypes. Note that the integer columns stayed integers despite the missing values, and that "NA" had to be declared as a missing value marker.

```
1 penguins = pl.read_csv("data/penguins.csv", null_values="NA"); penguins
```

py

shape: (344, 8)

species	island	bill_length_mm	...	body_mass_g	sex	year
str	str	f64	...	i64	str	i64
"Adelie"	"Torgersen"	39.1	...	3750	"male"	2007
"Adelie"	"Torgersen"	39.5	...	3800	"female"	2007
"Adelie"	"Torgersen"	40.3	...	3250	"female"	2007
...	...	...	...	...	...	...
"Chinstrap"	"Dream"	49.6	...	3775	"male"	2009
"Chinstrap"	"Dream"	50.8	...	4100	"male"	2009
"Chinstrap"	"Dream"	50.2	...	3775	"female"	2009

Subsetting

Without an index there is no `.loc` / `.iloc` distinction - `df[...]` takes column names or row positions and a slice is just a slice.

```
1 penguins["species"]
```

py

shape: (344,)

species
str
"Adelie"
"Adelie"
"Adelie"
...
"Chinstrap"
"Chinstrap"
"Chinstrap"

```
1 penguins[0:3, ["species", "island"]]
```

py

shape: (3, 2)

species	island
str	str
"Adelie"	"Torgersen"
"Adelie"	"Torgersen"
"Adelie"	"Torgersen"

```
1 penguins[0, "species"]
```

py

'Adelie'

Missing values

Missing data is `null` in every dtype (like R's `NA`) and `NaN` is a floating point value, distinct from missing.

Aggregation functions like `mean()` and `sum()` skip `null` by default and propagate `NaN`. Ordinary `min()` and `max()` ignore `NaN`; `nan_min()` and `nan_max()` propagate it.

```
1 s = pl.Series([1.0, None, np.nan])
```

```
1 s
```

shape: (3,)

	f64
0	1.0
1	null
2	NaN

```
1 s+1
```

shape: (3,)

	f64
0	2.0
1	null
2	NaN

```
1 s.is_null().to_list()
```

[False, True, False]

```
1 s.is_nan().to_list()
```

[False, None, True]

```
1 s.mean()
```

nan

```
1 s.fill_nan(None).mean()
```

1.0

Expressions

`pl.col("x")` does not return a column, it returns an *expression* - a description of a computation that is not attached to any DataFrame and has not been evaluated. Expressions compose with operators and methods and are only evaluated when handed to a context. This is polars' answer to data masking - build an object that represents the computation.

```
1 mass_kg = pl.col("body_mass_g") / 1000; mass_kg
```

py

```
[(col("body_mass_g")) / (dyn int: 1000)]
```

```
1 type(mass_kg)
```

py

```
<class 'polars.expr.expr.Expr'>
```

```
1 pl.col("body_mass_g").mean().alias("avg_mass")
```

py

```
col("body_mass_g").mean().alias("avg_mass")
```

Contexts

A context is a DataFrame method that evaluates expressions against its columns,

Context	Result	dplyr
<code>select()</code>	a new frame containing only these expressions	<code>select(), summarize()</code>
<code>with_columns()</code>	the frame with these expressions added	<code>mutate()</code>
<code>filter()</code>	the rows where these expressions are true	<code>filter()</code>
<code>group_by().agg()</code>	one row per group of these expressions	<code>group_by() > summarize()</code>

```
1 penguins.select(  
2   "species", mass_kg.alias("mass_kg")  
3 ).head(2)
```

py

shape: (2, 2)

species	mass_kg
str	f64
"Adelie"	3.75
"Adelie"	3.8

```
1 penguins.filter(  
2   pl.col("body_mass_g") > 6000  
3 ).select("species", "body_mass_g")
```

py

shape: (2, 2)

species	body_mass_g
str	i64
"Gentoo"	6300
"Gentoo"	6050

Combining predicates

Predicates work like pandas boolean masks: `&` and `|` with each comparison parenthesized, and `is_in()` in place of `isin()`. The one addition is that `filter()` accepts several predicates and ANDs them.

```
1 penguins.filter(  
2     pl.col("species").is_in(["Adelie", "Gento  
3     pl.col("body_mass_g") > 5000  
4 ).select(  
5     "species", "body_mass_g"  
6 ).head(3)
```

py

shape: (3, 2)

species	body_mass_g
str	i64
"Gentoo"	5700
"Gentoo"	5700
"Gentoo"	5400

```
1 penguins.filter(  
2     (pl.col("species") == "Gentoo")  
3     | (pl.col("body_mass_g") > 5000)  
4 ).select(  
5     "species", "body_mass_g"  
6 ).head(3)
```

py

shape: (3, 2)

species	body_mass_g
str	i64
"Gentoo"	4500
"Gentoo"	5700
"Gentoo"	4450

Expression expansion

One expression can expand to many columns - by listing names, by dtype, or with selectors (from `polars.selectors`). This is similar to dplyr's `across()`.

```
1 penguins.select(  
2     pl.col(  
3         "bill_length_mm",  
4         "bill_depth_mm"  
5     ).max()  
6 )
```

py

shape: (1, 2)

bill_length_mm	bill_depth_mm
f64	f64
59.6	21.5

```
1 penguins.select(  
2     pl.col(  
3         pl.String  
4     ).n_unique()  
5 )
```

py

shape: (1, 3)

species	island	sex
u32	u32	u32
3	3	3

```
1 import polars.selectors as cs  
2 penguins.select(cs.ends_with("_mm").mean().name.suffix("_mean"))
```

py

shape: (1, 3)

bill_length_mm_mean	bill_depth_mm_mean	flipper_length_mm_mean
f64	f64	f64
43.92193	17.15117	200.915205

group_by

`group_by().agg()` is the polars `group_by() |> summarize()` - each expression is evaluated once per group and the result has one row per group (in no guaranteed order). For a grouped mutate, add `.over("group")` to an expression and it is computed within each group but returns a value for every row.

```
1 mass = pl.col("body_mass_g")
2 (penguins
3   .group_by("species")
4   .agg(n = pl.len(), avg = mass.mean())
5   .sort("species")
6 )
```

py

shape: (3, 3)

species	n	avg
str	u32	f64
"Adelie"	152	3700.662252
"Chinstrap"	68	3733.088235
"Gentoo"	124	5076.01626

```
1 avg = mass.mean().over("species")
2 (penguins
3   .with_columns(rel = mass / avg)
4   .select("species", "rel")
5   .head(2)
6 )
```

py

shape: (2, 2)

species	rel
str	f64
"Adelie"	1.013332
"Adelie"	1.026843

Interoperability

Default `to_pandas()` converts nullable integer columns to floats. Arrow extension arrays preserve their integer types and nulls.

```
1 penguins.to_pandas().dtypes
```

py

```
species          str
island           str
bill_length_mm   float64
bill_depth_mm    float64
flipper_length_mm float64
body_mass_g      float64
sex              str
year             int64
dtype: object
```

```
1 penguins.to_pandas(
2     use_pyarrow_extension_array=True
3 ).dtypes
```

py

```
species          large_string[pyarrow]
island           large_string[pyarrow]
bill_length_mm   double[pyarrow]
bill_depth_mm    double[pyarrow]
flipper_length_mm int64[pyarrow]
body_mass_g      int64[pyarrow]
sex              large_string[pyarrow]
year             int64[pyarrow]
dtype: object
```

Exercise 1

Using polars and `data/flights.parquet`, repeat the questions from last lecture's pandas exercise. Questions 1 and 2 are the core; 3 and 4 are extensions if time allows.

1. How many flights to LAX did each legacy carrier (AA, UA, DL, US) have in May from JFK, and what was their mean actual air time (`air_time`)? Report only carriers with matching records.
2. Which plane (`tailnum`) has the most flight records from each New York airport? Exclude missing tail numbers and return all ties.
3. Which five calendar dates (`year`, `month`, `day`) had the lowest mean departure delay? Skip missing delays and break ties by earliest date.
4. Which flight has the largest arrival delay as a percentage of its actual air time (`100 * arr_delay / air_time`)? Exclude missing values and nonpositive air times, and return all ties.

Tidy data

Tidy data

country	year	cases	population
Afghanistan	1999	37745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	128042583

variables

country	year	cases	population
Afghanistan	1999	37745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	128042583

observations

country	year	cases	population
Afghanistan	1999	37745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	128042583

values

Tidy vs untidy

Happy families are all alike; every unhappy family is unhappy in its own way

— Leo Tolstoy, Anna Karenina

Is this data tidy? What are the variables, and what is an observation?

```
# A tibble: 317 × 7
  artist      track      date.entered  wk1  wk2  wk3  wk4
  <chr>      <chr>      <date>      <dbl> <dbl> <dbl> <dbl>
1 2 Pac      Baby Don't Cry (Keep... 2000-02-26      87    82    72    77
2 2Ge+her    The Hardest Part Of ... 2000-09-02      91    87    92    NA
3 3 Doors Down Kryptonite      2000-04-08      81    70    68    67
4 3 Doors Down Loser      2000-10-21      76    76    72    69
5 504 Boyz    Wobble Wobble      2000-04-15      57    34    25    17
6 98^0       Give Me Just One Nig... 2000-08-19      51    39    34    26
# i 311 more rows
```

Pivoting

Wide vs long

wide

id	x	y	z
1	a	c	e
2	b	d	f

Wide to long - tidyr

`pivot_longer()` stacks a set of columns into a pair of columns - one holding the old column names and one holding the values. The columns to pivot are chosen with tidy selection.

```
1 table4a
```

R

```
# A tibble: 3 × 3  
  country    `1999` `2000`  
  <chr>      <dbl> <dbl>  
1 Afghanistan    745   2666  
2 Brazil        37737  80488  
3 China         212258 213766
```

```
1 pivot_longer(  
2   table4a,  
3   cols = `1999`:`2000`,  
4   names_to = "year",  
5   values_to = "cases"  
6 )
```

R

```
# A tibble: 6 × 3  
  country    year  cases  
  <chr>      <chr> <dbl>  
1 Afghanistan 1999    745  
2 Afghanistan 2000   2666  
3 Brazil      1999  37737  
4 Brazil      2000  80488  
5 China       1999 212258  
6 China       2000 213766
```

Cleaning up while pivoting

Column names are strings, so the new names column is a character vector that often needs further work. `pivot_longer()` can strip a prefix, convert the type, and drop the structural missing values created by the wide layout.

```
1 billboard |>
2   pivot_longer(
3     starts_with("wk"),
4     names_to = "week", names_prefix = "wk", names_transform = as.integer,
5     values_to = "rank", values_drop_na = TRUE
6   )
```

```
# A tibble: 5,307 × 5
```

	artist	track	date.entered	week	rank
	<chr>	<chr>	<date>	<int>	<dbl>
1 2	Pac	Baby Don't Cry (Keep...	2000-02-26	1	87
2 2	Pac	Baby Don't Cry (Keep...	2000-02-26	2	82
3 2	Pac	Baby Don't Cry (Keep...	2000-02-26	3	72
4 2	Pac	Baby Don't Cry (Keep...	2000-02-26	4	77
5 2	Pac	Baby Don't Cry (Keep...	2000-02-26	5	87
6 2	Pac	Baby Don't Cry (Keep...	2000-02-26	6	94

```
# i 5,301 more rows
```

Wide to long - pandas & polars

pandas calls this `melt()` and polars `unpivot()`. Both name the columns to keep (`id_vars` / `index`) and pivot everything else unless told otherwise. `wide_pd` and `wide_pl` hold the same data as `table4a`.

```
1 wide_pd.melt(  
2     id_vars="country",  
3     var_name="year",  
4     value_name="cases"  
5 )
```

py

	country	year	cases
0	Afghanistan	1999	745
1	Brazil	1999	37737
2	China	1999	212258
3	Afghanistan	2000	2666
4	Brazil	2000	80488
5	China	2000	213766

```
1 wide_pl.unpivot(  
2     index="country",  
3     variable_name="year",  
4     value_name="cases"  
5 )
```

py

shape: (6, 3)

country	year	cases
str	str	i64
"Afghanistan"	"1999"	745
"Brazil"	"1999"	37737
"China"	"1999"	212258
"Afghanistan"	"2000"	2666
"Brazil"	"2000"	80488
"China"	"2000"	213766

Long to wide - tidyr

`pivot_wider()` is the inverse - the values of one column become new column names and the values of another fill them. The remaining columns (`id_cols`) identify the rows.

```
1 table2
```

A tibble: 12 × 4

	country	year	type	count
	<chr>	<dbl>	<chr>	<dbl>
1	Afghanistan	1999	cases	745
2	Afghanistan	1999	population	19987071
3	Afghanistan	2000	cases	2666
4	Afghanistan	2000	population	20595360
5	Brazil	1999	cases	37737
6	Brazil	1999	population	172006362

i 6 more rows

```
1 pivot_wider(  
2   table2,  
3   id_cols = c(country, year),  
4   names_from = type,  
5   values_from = count  
6 )
```

A tibble: 6 × 4

	country	year	cases	population
	<chr>	<dbl>	<dbl>	<dbl>
1	Afghanistan	1999	745	19987071
2	Afghanistan	2000	2666	20595360
3	Brazil	1999	37737	172006362
4	Brazil	2000	80488	174504898
5	China	1999	212258	1272915272
6	China	2000	213766	1280428583

Long to wide - pandas & polars

Both use `pivot()`. pandas moves the id columns into a row `MultiIndex` and names the column index after the pivoted column, polars returns ordinary columns.

`long_pd` and `long_pl` hold the same data as `table2`.

```
1 long_pd.pivot(  
2     index=["country", "year"],  
3     columns="type",  
4     values="count"  
5 )
```

py

type		cases	population
country	year		
Afghanistan	1999	745	19987071
	2000	2666	20595360
Brazil	1999	37737	172006362
	2000	80488	174504898
China	1999	212258	1272915272
	2000	213766	1280428583

```
1 long_pl.pivot(  
2     on="type",  
3     index=["country", "year"],  
4     values="count"  
5 )
```

py

shape: (6, 4)

country	year	cases	population
str	i64	i64	i64
"Afghanistan"	1999	745	19987071
"Afghanistan"	2000	2666	20595360
"Brazil"	1999	37737	172006362
"Brazil"	2000	80488	174504898
"China"	1999	212258	1272915272
"China"	2000	213766	1280428583

Duplicates and aggregation

If more than one row maps to the same cell a pivot wider is ambiguous - tidyR warns and returns list columns, pandas and polars error. Each has a way to aggregate as part of the pivot: `values_fn` in tidyR, `pivot_table()` in pandas, and `aggregate_function` in polars.

```
1 penguins_pd.pivot_table(  
2     index="island",  
3     columns="species",  
4     values="body_mass_g",  
5     aggfunc="mean"  
6 ).round(1)
```

species	Adelie	Chinstrap	Gentoo
island			
Biscoe	3709.7	NaN	5076.0
Dream	3688.4	3733.1	NaN
Torgersen	3706.4	NaN	NaN

```
1 penguins.pivot(  
2     on="species",  
3     index="island",  
4     values="body_mass_g",  
5     aggregate_function="mean"  
6 )
```

shape: (3, 4)

island	Adelie	Gentoo	Chinstrap
str	f64	f64	f64
"Torgersen"	3706.372549	null	null
"Biscoe"	3709.659091	5076.01626	null
"Dream"	3688.392857	null	3733.088235

The output columns of a pivot depend on the data, so a lazy query cannot know its schema in advance -

Exercise 2

The code below counts the penguins of each species measured on each island, species and island pairs that do not appear were never observed together.

```
1 palmerpenguins::penguins |>  
2   count(island, species)
```

```
# A tibble: 5 × 3  
  island    species     n  
  <fct>    <fct>   <int>  
1 Biscoe  Adelie    44  
2 Biscoe  Gentoo   124  
3 Dream   Adelie    56  
4 Dream   Chinstrap 68  
5 Torgersen Adelie    52
```

Using tidyr and then polars, construct a contingency table of these counts with islands as rows and species as columns, with zeros for the missing pairs.

Joins

Relational data

Tidy data is often spread across several tables, each describing one kind of thing - `nycflights13` has `flights`, `airlines`, `planes`, `airports`, and `weather`. Tables are connected by keys, columns whose values identify the matching rows in the other table. Two small tables make the behavior easy to see,

```
1 x = tibble(  
2   id = c(1, 2, 3),  
3   x = c("x1", "x2", "x3")  
4 )  
5 y = tibble(  
6   id = c(1, 2, 4),  
7   y = c("y1", "y2", "y4")  
8 )
```

R

```
1 x_pl = pl.DataFrame({  
2   "id": [1, 2, 3],  
3   "x": ["x1", "x2", "x3"]  
4 })  
5 y_pl = pl.DataFrame({  
6   "id": [1, 2, 4],  
7   "y": ["y1", "y2", "y4"]  
8 })
```

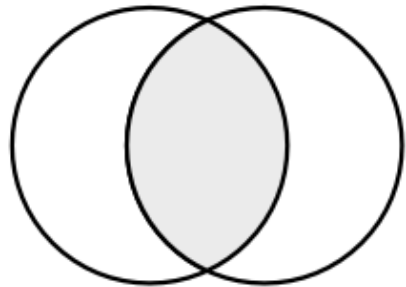
py

```
1 x_pd = x_pl.to_pandas()  
2 y_pd = y_pl.to_pandas()
```

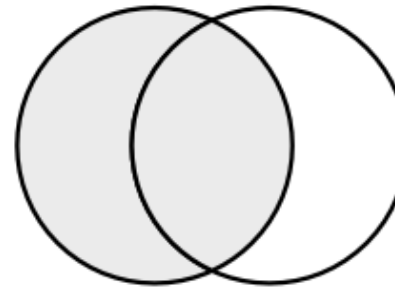
py

Join types

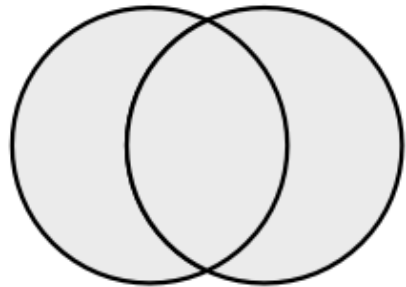
Mutating joins add the columns of y to x , and differ only in which unmatched rows are kept. Filtering joins keep or drop rows of x based on whether they have a match in y , without adding any columns.



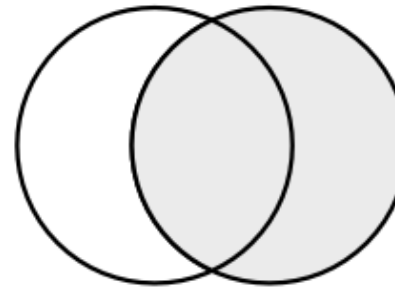
`inner_join(x, y)`



`left_join(x, y)`



`full_join(x, y)`



`right_join(x, y)`

dplyr

One function per join type, with keys given by `by`. If `by` is omitted all columns with common names are used, with a message. Unmatched rows are filled with `NA`.

```
1 left_join(x, y, by = "id")
```

R

```
# A tibble: 3 × 3
  id x      y
<dbl> <chr> <chr>
1     1 x1    y1
2     2 x2    y2
3     3 x3    <NA>
```

```
1 right_join(x, y, by = "id")
```

R

```
# A tibble: 3 × 3
  id x      y
<dbl> <chr> <chr>
1     1 x1    y1
2     2 x2    y2
3     4 <NA> y4
```

```
1 inner_join(x, y, by = "id")
```

R

```
# A tibble: 2 × 3
  id x      y
<dbl> <chr> <chr>
1     1 x1    y1
2     2 x2    y2
```

```
1 full_join(x, y, by = "id")
```

R

```
# A tibble: 4 × 3
  id x      y
<dbl> <chr> <chr>
1     1 x1    y1
2     2 x2    y2
3     3 x3    <NA>
4     4 <NA> y4
```

pandas

A single method, `merge()`, with the type given by `how` (default is `"inner"`). `indicator=True` adds a column recording where each row came from.

```
1 x_pd.merge(y_pd, on="id")
```

py

	id	x	y
0	1	x1	y1
1	2	x2	y2

```
1 x_pd.merge(y_pd, on="id", how="left")
```

py

	id	x	y
0	1	x1	y1
1	2	x2	y2
2	3	x3	NaN

```
1 x_pd.merge(  
2     y_pd, on="id", how="outer",  
3     indicator=True  
4 )
```

py

	id	x	y	_merge
0	1	x1	y1	both
1	2	x2	y2	both
2	3	x3	NaN	left_only
3	4	NaN	y4	right_only

polars

`join()` also takes the join method via `how` with a default of `"inner"`.

```
1 x_pl.join(y_pl, on="id", how="left")
```

py

shape: (3, 3)

id	x	y
i64	str	str
1	"x1"	"y1"
2	"x2"	"y2"
3	"x3"	null

```
1 x_pl.join(  
2     y_pl, on="id", how="full",  
3     coalesce=True  
4 )
```

py

shape: (4, 3)

id	x	y
i64	str	str
1	"x1"	"y1"
2	"x2"	"y2"
4	null	"y4"
3	"x3"	null

Keys with different names

Keys rarely share a name across tables. dplyr uses `join_by()`, pandas and polars use `left_on` and `right_on`. pandas keeps both key columns in the result, dplyr and polars keep only the left.

```
1 left_join(x, y2, by = join_by(id == key))
```

R

```
# A tibble: 3 × 3
```

	id	x	y
	<dbl>	<chr>	<chr>
1	1	x1	y1
2	2	x2	y2
3	3	x3	<NA>

```
1 x_pd.merge(  
2   y2_pd, how="left",  
3   left_on="id", right_on="key"  
4 )
```

py

	id	x	key	y
0	1	x1	1.0	y1
1	2	x2	2.0	y2
2	3	x3	NaN	NaN

```
1 x_pl.join(  
2   y2_pl, how="left",  
3   left_on="id", right_on="key"  
4 )
```

py

shape: (3, 3)

	id	x	y
	i64	str	str
1		"x1"	"y1"
2		"x2"	"y2"
3		"x3"	null

Duplicate keys

A row is repeated once for every match, so duplicated keys silently change the number of rows. If you expect a key to be unique, say so and the join will check - `relationship` in dplyr, `validate` in pandas and polars.

```
1 left_join(x, yd, by = "id")
```

R

A tibble: 4 × 3

	id	x	y
	<dbl>	<chr>	<chr>
1	1	x1	y1
2	1	x1	y2
3	2	x2	y3
4	3	x3	<NA>

```
1 left_join(  
2   x, yd, by = "id",  
3   relationship = "one-to-one"  
4 )
```

R

Error in `left\_join()`:

! Each row in `x` must match at most 1 row in `y`
i Row 1 of `x` matches multiple rows in `y`.

```
1 x_pl.join(yd_pl, on="id", how="left")
```

py

shape: (4, 3)

	id	x	y
	i64	str	str
1	1	"x1"	"y1"
1	1	"x1"	"y2"
2	2	"x2"	"y3"
3	3	"x3"	null

```
1 x_pl.join(  
2   yd_pl, on="id", how="left",  
3   validate="1:1"  
4 )
```

py

polars.exceptions.ComputeError: join keys did no

Filtering joins

`semi_join()` keeps the rows of `x` with a match in `y` and `anti_join()` the rows without one. polars has both as `how` options, pandas has neither and uses `isin()` instead.

```
1 semi_join(x, y, by = "id")
```

R

```
# A tibble: 2 × 2
```

```
  id x  
<dbl> <chr>  
1     1 x1  
2     2 x2
```

```
1 anti_join(x, y, by = "id")
```

R

```
# A tibble: 1 × 2
```

```
  id x  
<dbl> <chr>  
1     3 x3
```

```
1 x_pl.join(y_pl, on="id", how="anti")
```

py

```
shape: (1, 2)
```

id	x
i64	str
3	"x3"

```
1 x_pd[~x_pd["id"].isin(y_pd["id"])]
```

py

	id	x
2	3	x3

Missing keys

dplyr and pandas treat NA as equal to NA, so rows with missing keys are paired up even though nothing is known about either. polars follows SQL, where null is never equal to anything, so a row with a null key is unmatched.

```
1 a = tibble(k = c(1, NA), a = c("a1", "a2"))  
2 b = tibble(k = c(1, NA), b = c("b1", "b2"))
```

```
1 inner_join(a, b, by = "k")
```

```
# A tibble: 2 × 3  
      k a      b  
  <dbl> <chr> <chr>  
1     1 a1     b1  
2    NA a2     b2
```

```
1 a_pl = pl.DataFrame({"k": [1, None], "a": ["a1", "a2"]})  
2 b_pl = pl.DataFrame({"k": [1, None], "b": ["b1", "b2"]})  
3 a_pd, b_pd = a_pl.to_pandas(), b_pl.to_pandas()
```

```
1 a_pd.merge(b_pd, on="k")
```

```
      k  a  b  
0  1.0  a1 b1  
1  NaN  a2 b2
```

```
1 a_pl.join(b_pl, on="k")
```

shape: (1, 3)

k	a	b
i64	str	str
1	"a1"	"b1"

Missing keys - changing the default

dplyr switches to the SQL behavior with `na_matches = "never"` and polars to the dplyr behavior with `nulls_equal=True`. pandas has no equivalent, so the rows with missing keys must be dropped before merging.

```
1 inner_join(  
2   a, b, by = "k",  
3   na_matches = "never"  
4 )
```

```
# A tibble: 1 × 3  
      k a      b  
  <dbl> <chr> <chr>  
1     1 a1    b1
```

```
1 a_pl.join(b_pl, on="k", nulls_equal=True)
```

shape: (2, 3)

k	a	b
i64	str	str
1	"a1"	"b1"
null	"a2"	"b2"

```
1 a_pd.dropna(subset="k").merge(b_pd, on="k",
```

	k	a	b
0	1.0	a1	b1

The same choice applies to filtering joins - `anti_join()` with the default `na_matches` will not report a missing key as

Exercise 3

`data/planes.parquet` contains a record for each plane (`tailnum`) known to the FAA, including its `manufacturer`, `model`, and number of `seats`. Using `dplyr` and then `polars`, together with `data/flights.parquet`, answer the following.

1. How many flights have a tail number with no matching record in `planes`? Which carrier accounts for the most of them?
2. Which five manufacturers' planes flew the most flights out of New York?
3. Both tables have a `year` column. What happens to these columns in a join, and do they mean the same thing?

Summary

Three data frames

Concept	R / tibble	pandas	polars
column	atomic vector	<code>Series</code> (values + index)	<code>Series</code> (values only)
row labels	<code>row.names</code> , rarely used	<code>Index</code> , central	none
alignment	by position, with recycling	by index label	by position, lengths must match
mixed types	coerced to a common type	<code>object</code> dtype	error
missing values	<code>NA</code> in any type	<code>NaN</code> / <code>None</code> / <code>NaT</code> / <code>pd.NA</code> by dtype	<code>null</code> in any type
column references	data masking	strings, callables, <code>pd.col()</code>	<code>pl.col()</code> expressions
mutation	copy-on-modify	in place, copy-on-write for subsets	methods return new frames
grouped result	keys as columns	keys as (Multi)Index	keys as columns
laziness	none (dbplyr, dtplyr)	none	<code>LazyFrame</code> + optimizer
engine	C, single threaded	NumPy, single threaded	Rust + Arrow, multithreaded

Verbs

dplyr	pandas	polars
<code>filter(x > 1)</code>	<code>query("x > 1"), [pd.col("x") > 1]</code>	<code>filter(pl.col("x") > 1)</code>
<code>select(x, y)</code>	<code>[["x", "y"]], filter(regex=)</code>	<code>select("x", "y"), select(cs....())</code>
<code>mutate(z = x * 2)</code>	<code>assign(z = pd.col("x") * 2)</code>	<code>with_columns(z = pl.col("x") * 2)</code>
<code>arrange(desc(x))</code>	<code>sort_values("x", ascending=False)</code>	<code>sort("x", descending=True)</code>
<code>summarize(m = mean(x))</code>	<code>agg(m = ("x", "mean"))</code>	<code>select(m = pl.col("x").mean())</code>
<code>group_by(g)</code>	<code>groupby("g", as_index=False)</code>	<code>group_by("g")</code>
<code>mutate(m = mean(x), .by = g)</code>	<code>groupby("g")["x"].transform("mean")</code>	<code>pl.col("x").mean().over("g")</code>
<code>across(where(is.numeric), f)</code>	<code>select_dtypes("number").apply(f)</code>	<code>cs.numeric().f()</code>
<code>rename(new = old)</code>	<code>rename(columns={"old": "new"})</code>	<code>rename({"old": "new"})</code>
<code>left_join(y, by = "k")</code>	<code>merge(y, on="k", how="left")</code>	<code>join(y, on="k", how="left")</code>
<code>anti_join(y, by = "k")</code>	<code>[~pd.col("k").isin(y["k"])]</code>	<code>join(y, on="k", how="anti")</code>
<code>pivot_longer()</code>	<code>melt()</code>	<code>unpivot()</code>
<code>pivot_wider()</code>	<code>pivot(), pivot_table()</code>	<code>pivot()</code>

Takeaways

- polars drops the index and adds a strict schema, `null` for every dtype, expressions evaluated inside contexts, and a lazy mode with a query optimizer - much of which will look familiar again when we get to SQL and DuckDB.
- An expression is polars' answer to data masking - `pl.col()` builds an object describing a computation, and the same expression can be reused in `select()`, `with_columns()`, `filter()`, and `agg()`.
- Pivoting moves information between column names and cell values. The three libraries differ mostly in naming, but pandas returns the id columns as an index and a wider pivot fails on duplicates unless an aggregation is given.
- Joins differ in which unmatched rows they keep. Check the row count before and after, state the expected relationship so that duplicate keys are caught, and remember that polars does not match `null` keys while dplyr and pandas do.